Unit-I Topic-V Sorting and Searching

Introduction:

We know that, data structure is used to store information in structured manner. One of the common operations performed on the data are Sorting and Searching.

Definition:

"Sorting is the process of arranging the data in specific manner, whether in ascending order or descending order."

Advantage of sorting:

If the data is already sorted then searching particular element in sorted data is fast as compared to unsorted data. That is sorting makes searching fast.

Sorting techniques:

There are some following sorting techniques,

1) Exchange sort:

In this type of sorting, there is one condition and if condition is true then two elements are swapped (exchanged).

Following are some sorting methods that belong to exchange sort techniques.

a) Bubble Sort
b) Quick Sort
c) Selection Sort

Let's see these sorting methods in details-

I) Bubble sort:

Consider, there is an array 'X' having size 'n' and we have to sort it in ascending order by bubble sort method then 'n-1' passes or iterations are required.

Algorithm for bubble sort:

```
Step 1:     Start
Step 2:     Read 'n' elements of array 'X'
Step 3:     Initialize   i = 0
Step 4:     Initialize   j = 0
Step 5:  Check,
            If ( X[j] > X[j+1] )
            {
                Swap        X[j]    with   X[j+1]
            }

Step 6:     j = j+1
Step 7:     If    j <= n-2 then  goto     step 5
Step 8:     i = i +1
Step 9:     If    i<=n-1 then goto        step 4
Step 10:    Display 'n' elements of array 'X'
Step 11:    Stop.
```

Process to sort elements by using bubble sort method:

Pass - I: *I*n first pass, 0^{th} element is compared with 1^{st} element; if 0^{th} element is greater than 1^{st} element then they are exchanged. After that 1^{st} element is compared with 2^{nd} element; if 1^{st} element is greater than 2^{nd} element then they are exchanged. In the same way all elements are compared with their next elements and are exchanged if condition is true. And this is first pass.

After 1^{st} pass largest element is placed at the last index of array.

Pass - II: Similarly, in 2^{nd} pass, the comparisons are made from 0^{th} index up to 2^{nd} last index.

After 2^{nd} pass second largest element is placed at second last index or array.

Also, same process is happen in all passes.

To sort 'n' elements by using bubble sort, 'n-1' passes are required.

Eg. Consider, array 'X' we can sort it by using bubble sort method as fallow:

PASS- I

x	20	13	10	9	5	
	j	j+1				
x	13	20	10	9	5	
		j	j+1			
x	13	10	20	9	5	
			j	j+1		
x	13	10	9	20	5	
				j	j+1	
x	13	10	9	5	20	
					j	j+1

PASS- II

x	13	10	9	5	20	
	j	j+1				
x	10	13	9	5	20	
		j	j+1			
x	10	9	13	5	20	
			j	j+1		
x	10	9	5	13	20	
				j	j+1	
x	10	9	5	13	20	
					j	j+1

PASS- III

x	10	9	5	13	20	
	j	j+1				
x	9	10	5	13	20	
		j	j+1			
x	9	5	10	13	20	
			j	j+1		
x	9	5	10	13	20	
				j	j+1	
x	9	5	10	13	20	
					j	j+1

PASS- IV

x	9	5	10	13	20	
	j	j+1				
x	5	9	10	13	20	
		j	j+1			
x	5	9	10	13	20	
			j	j+1		
x	5	9	10	13	20	
				j	j+1	
x	5	9	10	13	20	
					j	j+1

After Pass- IV , We get

x	5	9	10	13	20

- To sort 'n' number, bubble sort method requires **n*(n-1)/2** maximum possible successful swapping to sort 'n' elements

Operation for bubble sort:

```
void    bubble( )
{
        int     x[100],n,i,j,k;
        cout<<"\n How many number you want to sort?";
        cin>>n;
        cout<<"\n Enter these elements : ";

        for( i = 0; i <= n-1; i++ )
        {
                cin>>x[i];
        }

        for( i=0; i <= n-1; i++ )     // total number of elements
        {
                for( j= 0 ; j<= n-2 ; j++ )     // total passes
                {
                        if( x[j] > x[j+1] )
                        {
                                k = x[j];
                                x[j] = x[j+1];
                                x[j+1] = k;
                        }
                }
        }

        cout<<"\nSorted array= ";

        for( i = 0; i <= n-1; i++ )
        {
                cout<<"\t"<<x[i];
        }
}
```

II) Quick sort [Partition exchanged sort]:

- The quick sort method is very efficient sorting method as compared to simple exchange sort and bubble sort methods.
- In this method, first we set 'pivot' element as 0^{th} index element and place that 'pivot' in an array in such a way that the elements belongs to left side of 'pivot' are less than 'pivot' and the elements belongs to right side of 'pivot' are greater than 'pivot'. That's why 'Quick sort' method is also called as 'Partition exchanged sort'.
- Quick sort uses divide and conquer strategy to sort entire data.

Algorithm:

Step 1: Start
Step 2: Read 'n' elements of array 'X'
Step 3: Initialize,
pivot = X[0]
down = 0
up = n-1
Step 4: Increment 'down' as long as *X[down] <= pivot* is true
Step 5: Decrement 'up' as long as *X[up] > pivot* is true

Step 6: check
If 'down' does not crosses 'up' then
Swap *X[up]* with *X[down]*

Step 7: Repeat the Step 4 to Step 6 till down<up is true
Step 8: If 'down' crosses 'up' then
Swap *X[up]* with *pivot*
Step 9: Now 'pivot' element is placed at proper position.
Here array is divided into two parts. (From left side of pivot and from right side of pivot)
Step 10: Now, do same process as mentioned above on left and right sub array. (Except pivot)
Step 11: Display 'n' elements of array.
Step 12: Stop

Eg. We can sort following numbers by using Quick Sort method as:

	0	1	2	3	4	5	6
X	23	18	48	32	7	55	85

→

Primary Setting:

Pivot= x[0]
down= 0
up = 6

	0	1	2	3	4	5	6
X	23	18	48	32	7	55	85
	Pivot/down						Up

We increment 'down' as long as x[down] <= pivot is true, therefore we get:

	0	1	2	3	4	5	6
X	23	18	48	32	7	55	85
	pivot		Down				Up

Now, we decrement 'up' as long as x[up] > pivot is true, therefore we get:

	0	1	2	3	4	5	6
X	23	18	48	32	7	55	85
	pivot		Down		Up		

Here, 'down' does not crosses 'up' therefore, we swap 'x[up]' with 'x[down]' & we get:

X	23	18	**7**	32	**48**	55	85
	pivot		Down		Up		

Still, here Down< up. Same processed is repeated as-
Again, we increment 'down' as long as x[down] <= pivot is true, therefore we get:

	0	1	2	3	4	5	6
X	23	18	7	32	48	55	85
	pivot			Down	Up		

Now, decrement 'up' as long as x[up] > pivot is true, therefore we get:

	0	1	2	3	4	5	6
X	23	18	7	32	48	55	85
	pivot		up	down			

Here, 'down' crosses 'up' therefore, we swap 'x[up]' with 'pivot' & we get sorted array:

	0	1	2	3	4	5	6
X	**7**	18	**23**	32	48	55	85

Note: Here, we got sorted array. But in some cases we not get sorted array after placing the pivot element. In such situation, separate quick sort process is applied on both portioned array (left partition array and left partition array)

Operation for quick sort method:

```
#include<iostream.h>
#include<conio.h>
int split(int[ ],int,int);
void quick(int[ ],int,int);
void main( )
{
   int  x[5], j;
   clrscr( );
   cout<<"\nEnter 5 numbers ";
   for(j=0;j<=4;j++)
   {
      cin>>x[j];
   }
   quick(x,0,4);
   cout<<"\nSorted Elements= ";
   for(int i=0;i<=4;i++)
   {
      cout<<"\t"<<x[i];
   }
   getch( );
}
```

```
void    quick(int z[ ],int lw,int  up)
{    int i;
     if(up>lw)
     {
       i=split(z, lw, up);
       quick(z, lw, i-1);
       quick(z, i+1, up);
     }
 }
int     split(int z[ ], int lw, int up)
{
     int     pivot, upper, lower, t;
     lower= lw;
     upper= up;
     pivot= z[lw];
     while(upper>lower)
     {
        while(z[lower]<=pivot)
        {
            lower++;
        }
        while(z[upper]>pivot)
        {
           upper--;
        }
        if(upper>lower)
        {
            t = z[lower];
            z[lower] = z[upper];
            z[upper] = t;
        }
     }
     t = z[lw];       //here, z[lw] is 'pivot' which is swapped with z[upper]
     z[lw] = z[upper];
     z[upper] = t;
     return(upper);
}
```

III) Selection sort:

We can sort 'n' elements of array 'X' in ascending order by using selection sort method as follows:

Process for selection sort:

Pass-I:

Search smallest element from X[0] to X[n-1].
Interchange X[0] with smallest element.
Result: X[0] is sorted.

Pass-II:

Search smallest element from X[1] to X[n-1].
Interchange X[1] with smallest element.
Result: X[1] is sorted.

Pass-III:

Search smallest element from X[2] to X[n-1].
Interchange X[2] with smallest element.
Result: X[2] is sorted.

Pass (n-1):

Search smallest element from X[n-2] to X[n-1].
Interchange X[n-2] with smallest element.
Result: X[n-2] is sorted.

Eg. We can sort following array by using Selection sort method as:

	0	1	2	3	4	5	6	7
X	75	35	42	13	87	24	64	57

Pass								
1	**75**	35	42	**13**	87	24	64	57
2	13	**35**	42	75	87	**24**	64	57
3	13	24	**42**	75	87	**35**	64	57
4	13	24	35	**75**	87	**42**	64	57
5	13	24	35	42	**87**	75	64	**57**
6	13	24	35	42	57	**75**	**64**	87
7	13	24	35	42	57	64	**75**	87

Operation for selection sort method:

```
void  selection( )
    {
        int i,j,k,pos,min,x[50],n;
            cout<<"\n How many number you want to sort?";
            cin>>n;
            cout<<"\n Enter these elements : ";

            for( i = 0; i <= n-1; i++ )
            {
                    cin>>x[i];
            }

             for(i=0;i<=n-1;i++)
             {
                              min=x[i];
                              for(j=i;j<=n-1;j++)
                              {
                               if(min>x[j])
                              {
                                min=x[j];
                              pos=j;
                              }
                             }
                               if(x[i]!=min)
                              {
                                      k=x[i];
                                      x[i]=x[pos];
                                      x[pos]=k;
                              }
             }
      cout<<"\n Sorted data=> ";
```

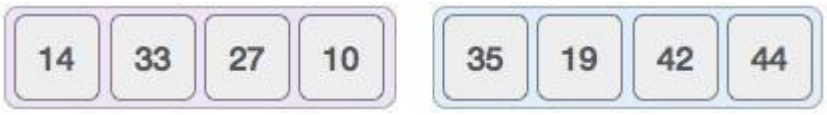
```
        for(i=0;i<=n-1;i++)
        {
          cout<<"\t"<<x[i];
        }
}
```

Merge Sort:

Merging is the process of combining two or more sorted data list into third data list in such a way that third data list becomes sorted.

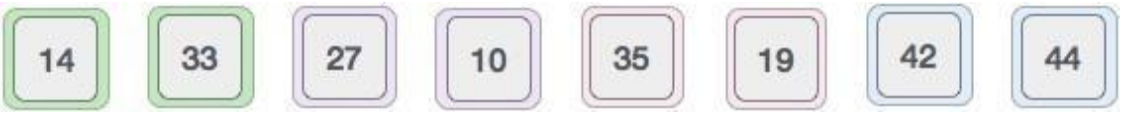
Eg. To understand merge sort, we take an unsorted array as the following −

We know that merge sort first divides the whole array iteratively into equal halves (parts) unless the atomic values are achieved. We see here that an array of 8 items is divided into two arrays of size 4.



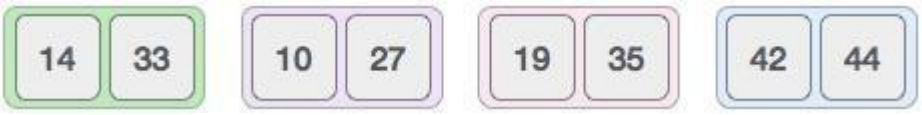
This does not change the sequence of appearance of items in the original. Now we divide these two arrays into halves.

We further divide these arrays and we achieve atomic value which can no more be divided.

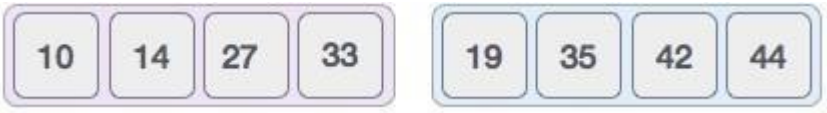


Now, we combine them in exactly the same manner as they were broken down.

We first compare the element for each list and then combine them into another list in a sorted manner. We see that 14 and 33 are in sorted positions. We compare 27 and 10 and in the target list of 2 values we put 10 first, followed by 27. We change the order of 19 and 35 whereas 42 and 44 are placed sequentially.



In the next iteration of the combining phase, we compare lists of two data values, and merge them into a list of found data values placing all in a sorted order.



After the final merging, the list should look like this −

Now we should learn some programming aspects of merge sorting as follows:

Algorithm

Merge sort keeps on dividing the list into equal halves until it can no more be divided. By definition, if it is only one element in the list, it is sorted. Then, merge sort combines the smaller sorted lists keeping the new list sorted too.

Step 1: Start
Step 2: Input 'n' elements in array say 'list'.
Step 3: Divide the list recursively into two halves until it can no more be divided.
Step 4: Merge the smaller lists into new list in sorted order.
Step 5: Display Sorted list.

Program:

```
#include<stdio.h>
#include<conio.h>
void mergesort(int [],int ,int );
void merge(int [],int,int,int,int);
void main( )
{
   clrscr( );
   int    a[30],n,i;
   cout<<"Enter no of elements:");
   cin>>n;
   cout<<"Enter array elements:");
      for(i=0;i<n;i++)
    {
      Cin>>a[i];
    }
   mergesort(a,0,n-1);
   cout<<"\nSorted array is :";
   for(i=0;i<n;i++)
    {
      cout<<"\t"<<a[i];
     }
```

```
void    merge(int a[],int  i1,int  j1,int  i2,int   j2)
{
   int    temp[50];   //array used for merging
   int    i,j,k;
   i=i1;    //beginning of the first list
   j=i2;    //beginning of the second list
   k=0;
 while(i<=j1 && j<=j2) //while elements in both lists
   {
      if(a[i]<a[j])
         temp[k++]=a[i++];
      else
         temp[k++]=a[j++];
   }

   while(i<=j1) //copy remaining elements of the first list
   {
      temp[k++]=a[i++];
    }
   while(j<=j2) //copy remaining elements of the second list
    {
```

```
    getch( );
}

void    mergesort(int a[],int i,int j)
{
    int     mid;
 if(i<j)
 {
   mid=(i+j)/2;
   mergesort(a,i,mid);      //left recursion
   mergesort(a,mid+1,j); //right recursion
   merge(a,i,mid,mid+1,j); //merging of two sorted sub-arrays
  }
}
```

```
        temp[k++]=a[j++];
    }

    //Transfer elements from temp[] back to a[]
   for(i=i1,j=0;i<=j2;i++,j++)
    {
          a[i]=temp[j];
     }

}
```

Note: Merge sort also uses divide and conquer strategy to sort the array.

Radix Sort:

Radix sort method is totally depending on number of digits and place of digit present in the number.

We know that any number can be combination of digits 0, 1, 2, …., 9 i.e. there are total 10 radices (digits).

Also, for implementation of radix sort method Queue is used.

Process:

Pass-I: In first pass, all the elements are placed in equivalent column according to their unit place digit.

Pass-II: In second pass, all the elements are placed in equivalent column according to their ten's place digit.

Pass-III: In third pass, all the elements are placed in equivalent column according to their hundred's place digit.

If data list (series of numbers) contains maximum number having 'n' digits then all elements of data list are get sorted in 'n' passes. i.e. it is totally depends on digits present in the number.

Eg. We can sort following array by using Radix sort method as:

	0	1	2	3	4	5	6
X	233	124	209	345	457	943	101

⇨ In given array, maximum number is 943 that contains only three digits therefore all elements are get sorted in tree passes.

In first pass we only consider unit place digit of each number, and place that number in equivalent column

Pass-I

Numbers	0	1	2	3	4	5	6	7	8	9
233				233						
124					124					
209										209
345						345				
457								457		
943				943						
101		101								

After Pass-I we get:

101	233	943	124	345	457	209

In Second pass we only consider ten's place digit of each number, and place that number in equivalent column

Pass-II

Numbers	0	1	2	3	4	5	6	7	8	9
101	101									
233				233						
943					943					
124			124							
345					345					
457						457				
209	209									

After Pass-II we get:

101	209	124	233	943	345	457

In third pass we only consider hundred's place digit of each number, and place that number in equivalent column

Pass-III

Numbers	0	1	2	3	4	5	6	7	8	9
101		101								
209			209							
124		124								
233			233							
943										943
345				345						
457					457					

After Pass-III we get:

101	124	209	233	345	457	943

Comparison of Sorting Methods:

- **Time complexity:** The amount of time taken by the algorithm or program for its execution is called 'Time complexity'
- **Space complexity:** The amount of memory space taken by the algorithm or program for its execution is called 'Space complexity'
- Following table shows time and space complexities of different sorting methods-

Sr.No	Sorting Method	Time Complexity	Space Complexity	Remark
1	Bubble sort	Best: O(n) Worst: $O(n^2)$	Additional space is required for only variables	---
2	**Quick sort**	Best: O (n log2 n) Worst: $O(n^2)$	Space depends on recursive calls or size of stack	Uses **Divide and conquer strategy**
3	Selection sort	$O(n^2)$	Additional space is required for only variables	Best & worst cases do not differ much.
4	**Merge Sort**	O (n log2 n)	Additional space is required for array	Uses **Divide and conquer** strategy.
5	**Radix Sort**	O(mn)	**Queues are required**	---

Searching:

"Searching is the process of finding particular element is present in data list or not."

- We can say that searching becomes successful if search element (Key value) is found in data list otherwise it is unsuccessful.
- Searching is applied on both kinds of data i.e. on sorted as well as unsorted data. But searching element in sorted data is faster and easy as compared to unsorted data. Thus, sorting makes searching fast.

There are two standard searching techniques as follows:

1) Linear (Sequential) Search.
2) Binary Search.

Let's see all these methods briefly…..

1) Linear (Sequential) Search:

The linear search technique is simplest & easy as compared to binary search method. In this type, searching is start from 0th index up to 'n-1' index of array (Here, 'n' is size of array).

Process for Linear (Sequential Search):

First search value (Key value) is compared with 0th index element of array, if Key value does not matches with 0th element then key value is compared with 1st element. Such type of comparison continues up to 'n-1' index of array until successful searching is done.

If element is matches with key value then it displays position of element in an array.

- Advantage of Linear Search:
  Linear search is applied on sorted as well as unsorted data.
- Disadvantage of Linear Search:
  Searching is start from 0th index up to 'n-1' index in sequential manner therefore linear search method is Slow as compared to binary search method.

Operation for linear Search for unsorted data:

```
void      linear_unsorted( )
{
 int      x[100], srch, i, f=0, n;
 cout<<"\n How many element you want?";
 cin>>n;
 cout<<"Enter these numbers");
 for(i=0;i<=n-1;i++)
 {
          cin>>x[i];
 }
 cout<<"\nEnter search element  ";
 cin>>srch;
 for(i=0;i<=n-1;i++)       // logic for linear search
{
   if( srch = = x[i] )
 {
   f = 1;
  break;
 }
}
if( f = = 1)
{
cout<<"\n Element is found at position= "<< i;
}
else
{
cout<<"\n Element is not found ";
}
}
```

Operation for linear Search for sorted data:

```
      void        linear_sorted( )
      {
                  int       x[100], srch, i, f=0, n;
                  cout<<"\n How many element you want?";
```

```
        cin>>n;
        cout<<"Enter these numbers");
        for(i=0;i<=n-1;i++)
        {
                cin>>x[i];
        }
        cout<<"\nEnter search element  ");
        cin>>srch;
                                                // bubble sort logic
        for( i=0; i <= n-1; i++ )    // total number of elements
        {
                for( j= 0 ; j<= n-2 ; j++ )          // total passes
                {
                        if( x[j] > x[j+1] )
                        {
                                k = x[j];
                                x[j] = x[j+1];
                                x[j+1] = k;
                        }
                }
        }

        cout<<"Sorted array=   ";

        for( i = 0; i <= n-1; i++ )
        {
                cout<<"\t"<<x[i];
        }


        for(i=0;i<=n-1;i++)                     // logic for linear search
        {
                if( srch = = x[i] )
                {
                        f = 1;
                        break;
                }
        }

        if( f = = 1)
        {
                cout<<"\n Element is found at position= "<< i;
        }
        else
        {
                cout<<"\n Element is not found ";
        }
}
```

Binary Search Technique:

Binary search method is very fast & efficient searching method as compared to linear search.

In binary search method, search key is only compared with middle element of array. That is search key not compared with every element of array that's why binary search method is fast as compared to linear search.

- Advantage of Binary Search:
  Binary search method is fast as compared to linear search because it requires less comparison to search the data.
- Disadvantage of Binary Search:
  Binary search method is only applicable over sorted data.

Algorithm for binary search method:

Step 1: Start
Step 2: Read 'n' elements of array 'X'
Step 3: Sort elements of array 'X' in ascending or descending order.
Step 4: Read the search value say 'srch'
Step 5: Initialize;
down = 0
up = n-1
f = 0
Step 6: Calculate;
mid = (down + up) / 2

Step 7: Check,
if (srch = = X[mid]) then
f = 1
and goto step 9
if (srch > X[mid])
down = mid + 1
if (srch < X[mid])
up = mid – 1
Step 8: Check,
if (down <= up) then
goto step 6
Step 9: Check,
if (f = = 1) then
Display— Element is found at position mid
Else
Display— Element is NOT found.
Step 10: Stop

Operation for binary Search method:

```
void    binary( )
{
        int     x[100], srch, i, f=0, down, up, n;
        cout<<"\n How many element you want?";
        cin>>n;
        cout<<"Enter these numbers";
        for(i=0;i<=n-1;i++)
        {
                cin>>x[i];
        }
        cout<<"\nEnter search element  ";
        cin>>srch;
                                                // bubble sort logic
        for( i=0; i <= n-1; i++ )    // total number of elements
```

```
        {
                for( j= 0 ; j<= n-2 ; j++ )          // total passes
                {
                        if( x[j] > x[j+1] )
                        {
                                k = x[j];
                                x[j] = x[j+1];
                                x[j+1] = k;
                        }
                }
        }
        cout<<"Sorted array=  ";
        for( i = 0; i <= n-1; i++ )
        {
                cout<<"\t"<<x[i];
        }
        down = 0;
        up = n-1;

        while( down <= up)                  //logic for binary search
        {
                mid = (down+up)/2;
                if(x[mid] = = srch)
                {
                        f = 1;
                        break;
                }
                else if( srch > x[mid])
                {
                        down = mid + 1;
                }
                else if( srch < x[mid])
                {
                        up = mid - 1;
                }
        }

        if( f = = 1)
        {
                cout<<"\n Element is found at position= "<< mid;
        }
        else
        {
                cout<<"\n Element is not found ";
        }
}
```

Theory Assignment

Q 1. What is sorting?

Q 2. Explain following sorting methods with algorithm, function and one example

1) Bubble Sort
2) Straight Selection sort
3) Partition exchanged sort (Quick sort)
4) Merge sort
5) Radix sort

Q 3. Sort following numbers in Descending order by bubble sort,Selection sort, merge sort, Radix sort methods (Show all passes)

I) 615,35,785,45,106,272,728,56,25,20

II) 50,827,215,99,44,51,98,14,5,212,163

III) 36,543,77,48,98,55,124,85,74,55,32

IV) 530,306,573,212,811,744,185,471,261,849

Q.4) Explain linear searching with its types.

Q.5) Explain Binary search method in detail.

Practical Assignment

1) Write a program to implement bubble sort method.
2) Write a program to implement quick sort method.
3) Write a program to implement selection sort method.
4) Write a program to implement merge sort method.
5) Write a program to implement linear search for unsorted data.
6) Write a program to implement linear search for sorted data.
7) Write a program to implement Binary search method.